

Documento de Selección del Enfoque del Proyecto

Sistema de Generación Óptima de Horarios Académicos

1. Introducción

El presente documento justifica las decisiones tecnológicas y metodológicas adoptadas para el desarrollo del Sistema de Generación Óptima de Horarios Académicos en Entornos de Currículo Flexible (SGOHA). La selección del enfoque se realizó evaluando alternativas concretas y contrastando su viabilidad técnica, alineación con el problema y restricciones del proyecto. Además, se consideraron aspectos como la escalabilidad y mantenibilidad de la solución propuesta. Asimismo, se buscó asegurar que las tecnologías elegidas permitan una futura evolución del sistema sin generar alta complejidad.

2. Criterios de Selección del Enfoque

Los criterios empleados para evaluar y comparar las alternativas fueron:

- **Viabilidad técnica:** capacidad del stack para soportar los requisitos funcionales y no funcionales del sistema.
- **Valor para el negocio:** facilidad de adopción, comunidad activa y curva de aprendizaje del equipo.
- **Gestión de riesgos:** madurez de las tecnologías, compatibilidad entre componentes y disponibilidad de documentación.
- **Alineación con metodología ágil (Scrum):** soporte para entregas incrementales y desarrollo iterativo.
- **Restricciones del proyecto:** tiempo de 12 semanas, equipo sin experiencia previa en plataformas complejas.

3. Stack Tecnológico Seleccionado: MERN

3.1 Descripción del Stack MERN

El stack MERN está compuesto por: MongoDB (base de datos NoSQL orientada a documentos), Express.js (framework web minimalista para Node.js), React.js (biblioteca frontend para construcción de interfaces reactivas), y Node.js (entorno de ejecución JavaScript del lado del servidor). Este stack permite utilizar JavaScript/TypeScript en toda la pila, reduciendo la fricción entre capas y unificando el lenguaje de desarrollo.

3.2 Comparación de Alternativas

Criterio	MERN (Seleccionado)	Django + React	Spring Boot + Angular
Lenguaje unificado	✓ JavaScript full-stack	✗ Python + JS	✗ Java + TS
Curva de aprendizaje	Media-baja	Media	Alta
Comunidad / Soporte	Muy alta	Alta	Alta
Escalabilidad	Alta (microservicios)	Media	Alta
Agilidad (Scrum)	Alta	Media	Media-baja
Viabilidad en 12 semanas	Alta	Media	Baja

3.3 Justificación por Componente

- **MongoDB**

Se selecciona MongoDB frente a PostgreSQL porque el modelo de datos de horarios académicos es flexible: los cursos, docentes y aulas tienen atributos variables según el contexto. MongoDB permite almacenar documentos JSON sin esquema rígido, facilitando iteraciones rápidas durante el desarrollo incremental.

- **Express.js**

Express.js proporciona una capa de servidor ligera sobre Node.js, ideal para construir APIs REST de forma rápida. Su minimalismo es adecuado para un PMV donde se priorizan funcionalidades core sobre complejidad de infraestructura.

- **React.js**

React.js permite construir interfaces SPA (Single Page Application) de forma dinámica, modular y basada en componentes reutilizables. Su ecosistema (React Router, hooks) es suficiente para cubrir los requerimientos de visualización de horarios sin necesidad de frameworks más complejos como Angular.

- **Node.js**

Node.js como entorno de ejecución permite manejar múltiples solicitudes concurrentes mediante su modelo event-driven, adecuado para el contexto de generación de horarios donde pueden existir consultas simultáneas de varios usuarios.

4. Metodología de Desarrollo: Scrum

Se adopta Scrum como marco de trabajo ágil por las siguientes razones:

- Los requerimientos del sistema son parcialmente definidos y evolucionan: Scrum permite incorporar cambios entre sprints.
- El proyecto tiene duración de 12 semanas, divisible en 3 sprints de aproximadamente 4 semanas cada uno (Sprint 0, Sprint 1, Sprint 2).
- Scrum promueve la colaboración activa, la revisión continua y la entrega incremental, alineándose con los criterios de evaluación del curso.
- Las ceremonias de Scrum (Sprint Planning, Daily Standup, Sprint Review, Retrospective) son viables de implementar con el equipo disponible.

Se descartó Kanban puro por carecer de sprints con objetivos delimitados, lo que dificulta la evidencia de evolución progresiva del PMV.

5. Flujo de Control de Versiones: Feature Branch Workflow

Se adopta Feature Branch Workflow por ser el enfoque más adecuado para equipos pequeños con experiencia media en Git:

- Cada funcionalidad se desarrolla en una rama dedicada (feature/nombre-funcionalidad).
- Los cambios se integran a main mediante Pull Requests con revisión de al menos un integrante.
- Se descartó Git Flow por su complejidad (ramas release y hotfix innecesarias para un PMV de 12 semanas).
- Se descartó Trunk-Based Development por requerir CI/CD robusto no previsto en el alcance.

6. Conclusión

El stack MERN con Scrum y Feature Branch Workflow representa la combinación técnica y metodológica más equilibrada para el contexto del proyecto: un equipo pequeño, restricciones de tiempo definidas, y la necesidad de construir un PMV funcional para la generación de horarios académicos. La decisión es coherente con los objetivos, las restricciones reales y los criterios de calidad del curso.